

# 프로세스 간 통신(IPC) 구현 — Part.1

## 발표자료 · 대본 합본

슬라이드 14매 + 슬라이드별 발표 대본

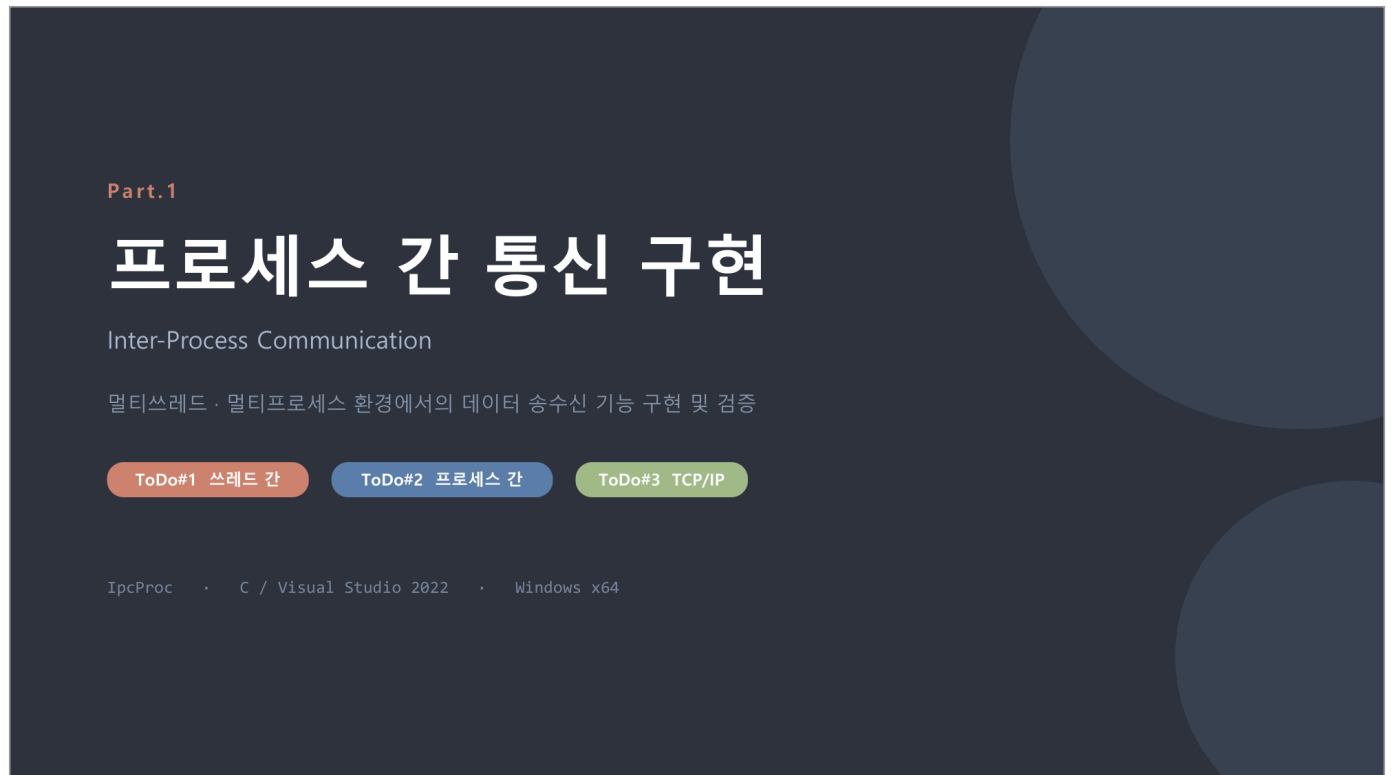
문서번호 IPCPROC-BRF-2026-001 (부속)

원본 자료 IPC\_구현\_발표자료.pptx (14매)

작 성 일 2026. 08. 28.

작 성 자 김장환

각 페이지는 슬라이드 이미지(상단)와 해당 슬라이드의 발표 대본(하단)으로 구성됩니다.



안녕하십니까. 지금부터 과제 Part.1, 프로세스 간 통신 IPC 구현 결과를 보고드리겠습니다.

이번 과제의 목표는 세 가지입니다. 첫째 스레드 간 송수신, 둘째 프로세스 간 송수신, 셋째 TCP/IP 를 이용한 다른 프로그램과의 송수신입니다. 세 가지 모두 구현을 완료하고 동일한 시나리오로 검증까지 마쳤습니다.

개발 환경은 Windows x64 와 Visual Studio 2022 이고, IPC 핵심 기능은 전부 C 로 작성했습니다. MFC 는 동작 확인용 UI 에만 사용했습니다.

## 목차

발표는 네 단계로 진행한다 — 아래 각 항목은 해당 슬라이드의 제목과 같다

### 1 과제 개요

슬라이드 3 ~ 4

과제 요구사항과 구현 매핑 / 솔루션 구조

### 2 구현

슬라이드 5 ~ 9

ToDo#1 쓰레드 간 송/수신 / ToDo#1 핵심 코드 / ToDo#2 프로세스 간 송/수신 / ToDo#3 TCP/IP / 리눅스 코드를 윈도우로 옮길 때

### 3 동작 확인

슬라이드 10 ~ 13

동작 확인용 UI / 동작 확인 · 프로세스 간 송/수신 / UI 정리 · New Process 버튼 통합 / 검증 결과

### 4 마무리

슬라이드 14

정리 — 배운 것 세 가지와 후속 조치 결과

발표 진행

총 14매 · 약 14분 + 질의응답 (시연 시 +3분) · 과제 개요 → 구현 → 동작 확인 → 마무리

발표 순서입니다. 전체는 네 단계로 진행합니다. 먼저 과제 개요에서 요구사항과 솔루션 구조를 보고, 구현에서 세 가지 ToDo 를 쓰레드, 프로세스, TCP/IP 순서로 설명드립니다. 이어서 동작 확인에서 UI 와 시연, 검증 결과를 보여드리고, 마지막에 정리로 마치겠습니다.

각 단계 아래에 적힌 항목이 그대로 슬라이드 제목이라, 자료를 다시 보실 때 목차만 보고 찾아가실 수 있습니다.

## 과제 요구사항과 구현 매핑

세 가지 ToDo 를 모두 같은 데이터 시나리오로 검증한다

### ToDo#1

#### 쓰레드 간 송/수신

전역변수 + 뮤텝스 / 세마포어

- 전역 링버퍼 8슬롯
- CRITICAL\_SECTION 1개
- 카운팅 세마포어 2개
- 송신·수신 쓰레드 각 1개

IpcThread.c

### ToDo#2

#### 프로세스 간 송/수신

메시지 큐 (Message Queue)

- 네임드 공유메모리 링버퍼
- 네임드 뮤텝스 1개
- 네임드 세마포어 2개
- 큐 이름으로 서로 참여

IpcMsgQ.c

### ToDo#3

#### 다른 프로그램과 송/수신

TCP / IP

- SocketUtility.h 윈도우 포팅
- Tx = 클라이언트 (connect)
- Rx = 서버 (bind-listen-accept)
- 함수·상수명 원본 유지

IpcSocket.c

### 공통 데이터 시나리오

송신측 1 로 초기화된 int 를 0.1초 간격으로 송신하고 송신 후 1씩 증가 (100까지)    수신측 받은 데이터를 출력

먼저 요구사항과 구현의 매핑입니다. ToDo 1번 쓰레드 간 통신은 전역 링버퍼에 뮤텝스와 세마포어를 조합해 구현했고, ToDo 2번 프로세스 간 통신은 메시지 큐로, ToDo 3번은 제공받은 SocketUtility 를 윈도우로 포팅해 TCP/IP 로 구현했습니다. 각각 IpcThread, IpcMsgQ, IpcSocket 이라는 C 파일 하나씩에 대응합니다.

이 장에서 한 가지만 짚겠습니다. 맨 아래 공통 데이터 시나리오입니다. 세 구현 모두 '1 로 초기화된 int 를 0.1초 간격으로 1씩 증가시키며 100까지 송신' 하는 같은 시나리오로 검증했습니다. 시나리오를 통일했기 때문에 뒤의 검증 결과에서 세 방식을 같은 기준으로 비교하실 수 있습니다.



## 솔루션 구조

IPC 기능은 전부 C로 작성한 DLL에 있고, MFC는 버튼과 로그 표시만 담당한다



빌드 결과 : build\x64\Debug\ → IpcUI.exe + IpcCore.dll

솔루션 구조입니다. IPC 기능은 전부 C로 작성한 IpcCore라는 DLL 안에 있고, MFC로 만든 IpcUI는 버튼과 로그 표시만 담당합니다. IPC 로직은 UI 쪽에 한 줄도 두지 않았습니다. 그래서 이 DLL은 UI 없이 콘솔 프로그램에 붙여도 그대로 동작합니다.

로그가 화면까지 오는 경로만 잠깐 설명드리겠습니다. 코어의 워커 쓰레드가 로그 함수를 호출하면 등록된 콜백이 불리는데, 이 콜백은 워커 쓰레드에서 실행되기 때문에 UI를 직접 건드리지 않습니다. PostMessage로 UI 쓰레드에 넘기기만 하고, 실제 리스트박스 출력은 UI 쓰레드가 합니다.

## ToDo#1 쓰레드 간 송/수신

lpcThread.c

전역 링버퍼를 뮤텍스로 보호하고, 빈 슬롯 / 찬 슬롯 개수를 세마포어 2개로 센다



세마포어를 함께 쓰는 이유

```
g_hSemEmpty 초기값 8 빈 슬롯 수
g_hSemFull 초기값 0 찬 슬롯 수

// 뮤텍스만 쓰면 '빌 때까지 대기'가
// 바쁜 대기가 되어 CPU 를 태운다
```

### 송신 / 수신 순서

**송신** sem\_wait(empty) → lock → write → unlock → sem\_post(full)

**수신** sem\_wait(full) → lock → read → unlock → sem\_post(empty)

Linux 대응 : CRITICAL\_SECTION = pthread\_mutex\_t CreateSemaphore = sem\_init \_beginthreadex = pthread\_create

ToDo 1번, 쓰레드 간 송수신입니다. 구조는 8슬롯짜리 전역 링버퍼이고, 인덱스와 데이터는 CRITICAL\_SECTION 하나로 보호합니다. 여기에 세마포어를 두 개 더 씁니다. 하나는 빈 슬롯 개수, 하나는 찬 슬롯 개수를 셉니다.

왜 뮤텍스만으로는 안 되는지가 이 장의 핵심입니다. 뮤텍스는 '동시에 못 들어가게' 하는 역할만 하지, '슬롯이 생길 때까지 기다리는' 것은 못 합니다. 뮤텍스만으로 흥내를 내면 락을 잡았다 놓았다를 반복하면서 CPU 를 태우는 바쁜 대기가 됩니다. 그래서 개수를 세면서 쓰레드를 채우고 깨우는 일은 세마포어가 맡습니다.

송신은 빈 슬롯을 기다렸다가 쓰고 찬 슬롯 카운트를 올리고, 수신은 정확히 그 반대로 동작합니다. 전형적인 생산자-소비자 패턴입니다.

## ToDo#1 핵심 코드

IpcThread.c

송신과 수신은 세마포어 두 개를 정확히 반대로 사용한다

```
f_IpcThreadQueueSend()

// ㉠ 빈 슬롯 대기 - 가득 차면 여기서 블록
if (WaitForSingleObject(g_hSemEmpty,
    uiTimeout_ms) != WAIT_OBJECT_0)
    return IPC_FAIL;

// ㉡ 뮉텍스로 링버퍼 상호배제
EnterCriticalSection(&g_stRingLock);
g_staRing[g_iRingTail] = *stpMsg;
g_iRingTail = (g_iRingTail + 1)
    % IPC_THREAD_RING_SLOTS;
g_iRingCount++;
LeaveCriticalSection(&g_stRingLock);

// ㉢ 찬 슬롯 +1 - 수신 스레드를 깨운다
ReleaseSemaphore(g_hSemFull, 1, NULL);
return IPC_PASS;
```

```
f_IpcThreadQueueRecv()

// ㉠ 찬 슬롯 대기 - 비면 여기서 블록
if (WaitForSingleObject(g_hSemFull,
    uiTimeout_ms) != WAIT_OBJECT_0)
    return IPC_FAIL;

// ㉡ 같은 뮉텍스로 상호배제
EnterCriticalSection(&g_stRingLock);
*stpMsg = g_staRing[g_iRingHead];
g_iRingHead = (g_iRingHead + 1)
    % IPC_THREAD_RING_SLOTS;
g_iRingCount--;
LeaveCriticalSection(&g_stRingLock);

// ㉢ 빈 슬롯 +1 - 송신 스레드를 깨운다
ReleaseSemaphore(g_hSemEmpty, 1, NULL);
return IPC_PASS;
```

세마포어 대기를 뮉텍스 밖에 두는 것이 핵심 — 잠근 채로 기다리면 상대가 들어올 수 없어 데드락이 된다

핵심 코드를 송신과 수신 나란히 놓고 보겠습니다. 송신은 세 단계입니다. 첫째 빈 슬롯 세마포어를 기다립니다. 큐가 가득 차 있으면 여기서 재워집니다. 둘째 뮉텍스를 잡고 tail 위치에 쓰고 인덱스를 돌립니다. 셋째 찬 슬롯 세마포어를 올려서 수신 스레드를 깨웁니다. 수신은 세마포어 두 개를 정확히 반대로 쓰는 대칭 구조입니다.

이 슬라이드에서 하나만 기억하신다면 맨 아래 문장입니다. 세마포어 대기는 반드시 뮉텍스 밖에 둔다. 만약 락을 잡은 채로 슬롯을 기다리면, 그 슬롯을 만들어 줄 상대 스레드가 락에 막혀서 못 들어오고, 서로 영원히 기다리는 데드락이 됩니다.

## ToDo#2 프로세스 간 송/수신

lpcMsgQ.c

Windows 에는 System V 메시지 큐가 없어 같은 동작을 직접 만들었다

| Linux                               | Windows 구현                                             |
|-------------------------------------|--------------------------------------------------------|
| <code>msgget(key, IPC_CREAT)</code> | <code>CreateFileMapping</code> 네임드 공유메모리 + 링버퍼 헤더 초기화  |
| <code>msgsnd()</code>               | 세마포어(빈슬롯) 대기 → 뮤텍스 → <code>write</code> → 세마포어(찬슬롯) 증가 |
| <code>msgrcv()</code>               | 세마포어(찬슬롯) 대기 → 뮤텍스 → <code>read</code> → 세마포어(빈슬롯) 증가  |
| <code>msgctl(IPC_RMID)</code>       | 모든 핸들 <code>CloseHandle</code>                         |
| <code>msgtype</code>                | <code>st_lpcMsg.iMsgType</code>                        |

### 커널 오브젝트 이름

|                                              |       |
|----------------------------------------------|-------|
| <code>Local\IpcProc.&lt;큐이름&gt;.map</code>   | 공유메모리 |
| <code>Local\IpcProc.&lt;큐이름&gt;.mtx</code>   | 뮤텍스   |
| <code>Local\IpcProc.&lt;큐이름&gt;.sem.e</code> | 빈 슬롯  |
| <code>Local\IpcProc.&lt;큐이름&gt;.sem.f</code> | 찬 슬롯  |

### 기동 순서에 무관하게 만든 방법

- `f_lpcMsgQCreate()` 는 없으면 만들고 있으면 참여한다
- 링버퍼 헤더 초기화는 뮤텍스 안에서 매직값으로 한 번만
- 세마포어 초기 카운트가 최초 생성 시에만 반영되는 Windows 동작을 그대로 이용

GlobalW 이 아니라 LocalW 을 쓰므로 관리자 권한이 필요 없다

ToDo 2번, 프로세스 간 통신입니다. 리눅스라면 System V 메시지 큐, `msgget`·`msgsnd`·`msgrcv` 를 쓰면 되는데 Windows 에는 같은 것이 없습니다. 그래서 표와 같이 항목별로 대응시켜 같은 의미를 직접 만들었습니다. `msgget` 은 네임드 공유메모리 생성으로, `msgsnd` 와 `msgrcv` 는 방금 스레드에서 본 것과 같은 '세마포어 대기, 뮤텍스, 읽고 쓰기' 절차로 대응됩니다.

정리하면 ToDo 1 과 완전히 같은 구조를, 커널 오브젝트에 이름을 붙여서 프로세스 경계 너머로 확장한 것입니다.

실무적으로 신경 쓴 부분은 기동 순서입니다. Create 함수는 큐가 없으면 만들고 있으면 참여합니다. 링버퍼 헤더 초기화는 뮤텍스 안에서 매직값 검사로 한 번만 하고, 세마포어 초기 카운트가 최초 생성자에게만 반영되는 Windows 동작을 그대로 이용했습니다. 그래서 수신을 먼저 띄우든 송신을 먼저 띄우든 동작합니다.

## ToDo#3 TCP / IP

lpcSocket.c

첨부받은 Linux SocketUtility 를 Winsock2 로 포팅했다 — 함수명 · 인자 · 상수는 원본 그대로

### Tx 송신 = 클라이언트

socket → connect → send

- 블로킹 connect 는 실패 확정까지 20초 이상 걸린다
- 논블로킹 + select 로 1초마다 재시도하게 바꿨다

### Rx 수신 = 서버

socket → bind → listen → accept → recv

- accept 를 200ms 단위 select 루프로 잘게 나눴다
- 덕분에 [Stop] 을 누르면 대기 중에도 바로 빠져나온다

```
데모 송신 - 프레임 헤더 없이 int 4바이트
for (iData = IPC_DEMO_FIRST_VALUE;
    iData <= IPC_DEMO_LAST_VALUE; iData++) {
    iWire = g_iDemoUseNBO
        ? htonl(iData) : iData;
    f_SocketSendTCP_IPv4(&g_stDemoSocket,
        &iWire, sizeof(iWire));
    // 0.1초 대기 (정지 요청도 함께 확인)
    s_IsDemoStopRequested(100);
}
```

### 바이트 오더

- x64 Windows 와 x86-64 Linux 는 둘 다 little-endian 이라 기본값으로 값이 그대로 맞는다
- 상대가 htonl 을 쓰면 UI 의 htonl 체크박스를 켜서 맞춘다
- 송/수신은 요청한 크기를 다 처리할 때까지 반복한다 (>0 처리 바이트, 0 타임아웃, -1 상대 종료)

ToDo 3번, TCP/IP 입니다. 제공받은 리눅스 SocketUtility 를 Winsock2 로 포팅했고, 함수명과 인자, 상수는 원본을 그대로 유지했습니다. 역할도 원본 규약대로 송신이 클라이언트로 connect 하고, 수신이 서버로 bind, listen, accept 합니다.

포팅하면서 동작을 손 본 곳이 두 군데 있습니다. 첫째, 블로킹 connect 는 상대가 없을 때 실패 확정까지 20초 이상 걸립니다. 그래서 논블로킹으로 바꾸고 select 로 1초씩 끊어서 재시도하게 했습니다. 둘째, accept 대기를 200 밀리초 단위 select 루프로 잘게 나눴습니다. 덕분에 서버가 접속을 기다리는 중이라도 Stop 버튼에 바로 반응합니다.

데이터는 프레임 헤더 없이 int 4바이트를 그대로 보냅니다. x64 윈도우와 리눅스는 둘 다 little-endian 이라 기본 설정으로도 값이 맞고, 상대가 htonl 을 쓰는 경우는 UI 의 체크박스로 맞출 수 있게 했습니다.

## 리눅스 코드를 윈도우로 옮길 때

컴파일은 통과하는데 런타임에 이상 동작하는 항목이 특히 위험하다

| 항목             | Linux                            | Windows                                |
|----------------|----------------------------------|----------------------------------------|
| 소켓 핸들          | int                              | SOCKET (UINT_PTR) — x64 에서 64bit       |
| 송/수신 타임아웃      | setsockopt(SO_RCVTIMEO, timeval) | DWORD 밀리초 — <b>timeval 을 넘기면 엉뚱한 값</b> |
| 소켓 닫기          | close()                          | closesocket()                          |
| 초기화            | 없음                               | WSAStartup / WSACleanup 필요             |
| 에러 확인          | errno                            | WSAGetLastError()                      |
| select() 1번 인자 | maxfd + 1                        | 무시된다                                   |
| 마이크로초 대기       | usleep()                         | 없음 — Sleep 또는 QPC 스핀                   |

타임아웃 값은 형이 맞아 보여서 그냥 넘어가기 쉽다 — timeval(초:마이크로초) 을 DWORD(밀리초) 로 변환하는 코드를 내부에 두었다

포팅 주의점을 표로 정리했습니다. 이 중에 정말 위험한 것은 컴파일이 그냥 통과되는 항목들입니다. 대표적인 것이 소켓 타임아웃입니다. 리눅스는 timeval 구조체를 받는데 윈도우는 DWORD 밀리초를 받습니다. 형만 맞추면 컴파일이 되기 때문에 엉뚱한 타임아웃이 걸린 채로 조용히 지나갑니다. 그래서 timeval 을 밀리초로 변환하는 코드를 라이브러리 내부에 넣어 두었습니다.

그 외에 소켓 핸들이 int 가 아니라 SOCKET 타입이고 x64 에서는 64비트라는 점, close 대신 closesocket, 시작할 때 WSAStartup 이 필요하다는 점, 에러는 errno 가 아니라 WSAGetLastError 로 확인한다는 점을 표에서 보실 수 있습니다.

## 동작 확인용 UI

버튼 하나가 코어 함수 하나를 호출한다 — 모든 로그는 아래 리스트박스에 모인다

The screenshot shows the 'IpcProc' application window. It has a title bar with a minus, maximize, and close button. The window is divided into several sections:

- Thread:** Contains 'Start' and 'Stop' buttons. A red circle '1' is next to the 'Start' button.
- Message Queue:** Contains a 'Name' input field with 'IpcDemoQ', and 'Recv', 'Send', and 'Stop' buttons. A blue circle '2' is next to the 'Stop' button.
- TCP/IP:** Contains 'IP' (127.0.0.1), 'Port' (51000), and a checkbox for 'htonl'. It also has 'Recv', 'Send', and 'Stop' buttons. A green circle '3' is next to the 'Stop' button.
- Log List:** A list box containing log entries like '14:32:10 [TH] rx 38', '14:32:10 [TH] tx 39', etc. A blue circle '4' is next to the list box.
- Buttons:** At the bottom, there are 'New Process', 'Clear', and 'Close' buttons. A red circle '5' is next to the 'New Process' button.

The status bar at the bottom shows 'IpcUI.exe · Debug | x64'.

### 1 Thread

[Start] 로 송신-수신 쓰레드 한 쌍을 기동한다. 동작 중에는 [Start] 가 비활성으로 바뀐다

### 2 Message Queue

큐 이름을 맞춘 두 프로세스가 각각 [Recv] / [Send]. 동작 중에는 이름 입력칸이 잠긴다

### 3 TCP/IP

IP · Port · htonl 을 정하고 한쪽은 [Recv](서버), 다른쪽은 [Send](클라이언트)

### 4 로그 리스트박스

세 채널의 로그가 시각과 함께 한 곳에 쌓인다. 최대 2000줄을 유지한다

### 5 New Process

같은 exe 를 인자 없이 한 번 더 띄운다. 프로세스 간 확인에 쓴다

동작 확인용 UI 입니다. 버튼 하나가 코어 함수 하나를 그대로 호출하는 구조입니다. 위에서부터 쓰레드, 메시지 큐, TCP/IP 세 구역이고, 세 채널의 로그는 아래 리스트박스 한 곳에 시각과 함께 쌓입니다. 동작 중에는 해당 구역의 시작 버튼과 입력칸이 잠겨서 잘못 누르는 것을 막습니다.

## 동작 확인 · 프로세스 간 송/수신

ToDo#2

[New Process] 로 창을 하나 더 띄우고 한쪽은 [Recv], 다른쪽은 [Send] 를 누른다

**수신 프로세스**      먼저 띄운 창 · [Recv]

**IpcProc**

Thread

Start Stop

Message Queue

Name IpcDemoQ Recv Send Stop

TCP/IP

IP 127.0.0.1 Port 51000 ☐ htonl Recv Send Stop

14:35:08 [MQ] rx 15  
14:35:08 [MQ] rx 16  
14:35:08 [MQ] rx 17  
14:35:08 [MQ] rx 18  
14:35:08 [MQ] rx 19  
14:35:08 [MQ] rx 20  
14:35:08 [MQ] rx 21  
14:35:09 [MQ] rx 22  
14:35:09 [MQ] rx 23  
14:35:09 [MQ] rx 24  
14:35:09 [MQ] rx 25  
14:35:09 [MQ] rx 26  
14:35:09 [MQ] rx 27  
14:35:09 [MQ] rx 28  
14:35:09 [MQ] rx 29

New Process Clear Close

**송신 프로세스**      [New Process] 로 띄운 창 · [Send]

**IpcProc**

Thread

Start Stop

Message Queue

Name IpcDemoQ Recv Send Stop

TCP/IP

IP 127.0.0.1 Port 51000 ☐ htonl Recv Send Stop

14:35:08 [MQ] tx 15  
14:35:08 [MQ] tx 16  
14:35:08 [MQ] tx 17  
14:35:08 [MQ] tx 18  
14:35:08 [MQ] tx 19  
14:35:08 [MQ] tx 20  
14:35:09 [MQ] tx 21  
14:35:09 [MQ] tx 22  
14:35:09 [MQ] tx 23  
14:35:09 [MQ] tx 24  
14:35:09 [MQ] tx 25  
14:35:09 [MQ] tx 26  
14:35:09 [MQ] tx 27  
14:35:09 [MQ] tx 28  
14:35:09 [MQ] tx 29

New Process Clear Close

같은 큐 이름 **IpcDemoQ** 만 맞으면 어느 쪽을 먼저 띄워도 된다 — `f_IpcMsgQCreate()` 가 없으면 만들고 있으면 참여하기 때문

프로세스 간 확인 절차입니다. New Process 버튼으로 같은 프로그램을 하나 더 띄우고, 먼저 띄운 창에서 Recv, 새로 띄운 창에서 Send 를 누릅니다. 그러면 수신 창에 1 부터 100 까지 차례로 올라옵니다.

큐 이름 IpcDemoQ 만 같으면 어느 쪽을 먼저 시작해도 됩니다. 앞에서 말씀드린 '없으면 만들고 있으면 참여' 하는 Create 동작 덕분입니다.



## UI 정리 · New Process 버튼 통합

역할별로 흩어져 있던 버튼을 공용 버튼 하나로 줄여 과제의 핵심 기능만 남겼다

### 변경 전

- Message Queue 와 TCP/IP 에 [New Process] 버튼이 각각 하나씩
- 버튼이 상대 역할을 스스로 정해 명령행으로 넘겼다
- 새 창이 그 역할을 자동으로 시작했다

```
CIpcUIDlg::RunPeer()
strCmd.Format(_T("\\%s\\" /peer:%s /name:%s"
"/ip:%s /port:%u%s"), szExePath,
lpzRole, m_strQueueName, m_strIPAddr,
m_nPortNum, m_bNetworkByteOrder
? _T(" /nbo") : _T(""));

// + ParseIpcArguments() 명령행 파서
// + OnInitDialog 작동 시작 분기
```

### 변경 후

- 공용 [New Process] 버튼 하나만 하단에 둔다
- 인자 없이 같은 exe 를 한 번 더 띄우는 것이 전부
- 역할은 사용자가 새 창에서 직접 누른다

```
CIpcUIDlg::OnBnClickedNewProcess()
TCHAR szExePath[MAX_PATH] = { 0 };
GetModuleFileName(nullptr, szExePath,
MAX_PATH);

CreateProcess(szExePath, nullptr, nullptr,
nullptr, FALSE, 0, nullptr, nullptr,
&si, &pi);
```

**결과** 버튼 2개 · 명령행 파서 · 자동 시작 분기 제거 → UI 코드 84줄 감소, 과제가 요구한 IPC 기능은 그대로

UI 를 정리한 내용도 짧게 보고드립니다. 원래는 메시지 큐와 TCP 구역에 New Process 버튼이 따로 있었고, 새 창에 역할을 명령행 인자로 넘겨 자동 시작까지 했습니다. 그런데 이것은 과제 요구사항 밖의 편의 기능이고 검증할 코드만 늘리는 것이라 전부 걷어냈습니다.

지금은 인자 없이 창만 하나 더 띄우는 공용 버튼 하나만 있고, 역할은 사용자가 새 창에서 직접 누릅니다. UI 코드가 84줄 줄었고, 과제가 요구한 IPC 기능은 그대로입니다.

## 검증 결과

세 가지 모두 1 부터 100 까지 0.1초 간격으로 송신하고 수신측이 같은 값을 출력한다

### 쓰레드 간

ToDo#1

```
[TH] start
[TH] tx 1
[TH] rx 1
[TH] tx 2
[TH] rx 2
[TH] tx 3
[TH] rx 3
...
[TH] tx 100
[TH] rx 100
[TH] tx done (100)
[TH] rx done (100)
```

### 프로세스 간

ToDo#2

```
[MQ] queue 'IpcDemoQ' open
[MQ] start (rx)
[MQ] rx 1
[MQ] rx 2
[MQ] rx 3
...
[MQ] rx 99
[MQ] rx 100
[MQ] stop
[MQ] rx done (100)
```

### TCP / IP

ToDo#3

```
[TCP] listen 0.0.0.0:51000
[TCP] accept 127.0.0.1:60312
[TCP] rx 1
[TCP] rx 2
[TCP] rx 3
...
[TCP] rx 99
[TCP] rx 100
[TCP] peer closed
[TCP] rx done (100)
```

### 확인한 것

- ① 100건이 순서대로 빠짐없이 도착한다
- ② 수신측이 대기 중에도 [Stop] 에 바로 반응한다
- ③ 두 프로세스의 기동 순서를 바꿔도 동작한다

검증 결과입니다. 세 방식 모두 같은 시나리오로 돌렸고 로그 형식도 같아서 나란히 비교할 수 있습니다. 확인한 것은 세 가지입니다. 첫째, 1 부터 100 까지 백 건이 순서대로 빠짐없이 도착합니다. 둘째, 수신측이 대기 중이어도 Stop 에 바로 반응합니다. 셋째, 두 프로세스의 기동 순서를 바꿔도 동작합니다.

## 정리

### 1 뮤텍스와 세마포어는 짝이다

상호배제는 뮤텍스, 개수를 세고 기다리는 것은 세마포어. 하나로 다른 하나를 흉내내면 바쁜 대기나 데드락이 된다

### 2 메시지 큐는 없으면 만들 수 있다

공유메모리 링버퍼 + 네임드 뮤텍스 + 세마포어 2개면 msgsnd / msgrcv 와 같은 의미를 그대로 만들 수 있다

### 3 포팅에서 위험한 것은 타입이 맞는 차이

SO\_RCVTIMEO 처럼 컴파일은 통과하는데 값의 의미가 달라지는 항목을 먼저 찾아야 한다

후속 조치 완료 · \_Sync 계열 핸드셰이크를 원본 SocketUtility.c 와 동일한 UDP 사이드채널 절차로 재구현하고 루프백 왕복 시험까지 통과했다

정리하겠습니다. 이번 과제에서 얻은 것은 세 가지입니다. 첫째, 뮤텍스와 세마포어는 짝입니다. 상호배제는 뮤텍스가, 개수를 세며 기다리는 것은 세마포어가 맞습니다. 하나로 다른 하나를 흉내내면 바쁜 대기나 데드락이 됩니다. 둘째, 메시지 큐는 없으면 만들 수 있습니다. 공유메모리 링버퍼와 네임드 뮤텍스, 세마포어 두 개면 msgsnd, msgrcv 와 같은 의미를 그대로 만들 수 있습니다. 셋째, 포팅에서 정말 위험한 것은 타입이 맞아 보이는 차이입니다. 컴파일러가 잡아주지 않기 때문입니다.

슬라이드 하단의 후속 조치입니다. \_Sync 계열 핸드셰이크는 원본 SocketUtility.c 를 확보한 뒤 원본과 동일한 UDP 사이드채널 절차로 재구현을 완료했고, 루프백 왕복 시험까지 통과했습니다.

이상으로 발표를 마치겠습니다. 질문 주시면 답변드리겠습니다.